

数据库应用系统综合设计实验报告

实验名称： 存储过程和触发器 .

实验类型： 设计型实验

指导教师： 陈骏

专业班级： _____

姓 名： _____

学 号： _____

联系电话： _____

成绩： _____

1. 实验目的

- 了解存储过程的概念、优点
- 熟练掌握创建存储过程的方法
- 熟练掌握存储过程的调用方法
- 了解触发器的概念、优点
- 掌握触发器的方法和步骤
- 掌握触发器的使用

2. 实验环境

oracle Database 19c 企业版、(64 位) PLSQL Developer 12、Windows 10

3. 实验内容

1、借书功能

建立存储过程完成图书管理系统中的借书功能，并调用该存储过程实现借书功能。功能要求：

- 借书时要求输入借阅流水号，借书证号，图书编号。(即该存储过程有 3 个输入参数)
- 借书时，借书日期为系统时间。
- 图书的是否借出改为‘是’

```
CREATE OR REPLACE PROCEDURE 借书 (  
    -- 输入参数  
    b_借阅流水号 IN 借阅.借阅流水号%TYPE,  
    b_借书证号 IN 借阅.借书证号%TYPE,  
    b_图书编号 IN 借阅.图书编号%TYPE  
)  
AS  
    b_是否借出 图书.是否借出%TYPE; -- 临时变量  
BEGIN  
    SELECT 是否借出 INTO b_是否借出  
    FROM 图书  
    WHERE 图书编号 = b_图书编号; -- 判断图书是否已经借出  
    IF b_是否借出 = '是' THEN  
        RAISE APPLICATION_ERROR(-20001, '该图书已被借出, 无法再次借阅'); -- 插入借阅记录  
    ELSE  
        INSERT INTO 借阅 (借阅流水号, 借书证号, 图书编号, 借书日期, 归还日期, 罚款分类号, 备注)  
        VALUES (b_借阅流水号, b_借书证号, b_图书编号, SYSDATE, NULL, NULL, NULL);  
        UPDATE 图书  
        SET 是否借出 = '是'  
        WHERE 图书编号 = b_图书编号; -- 更图书是否借出改为“是”  
        COMMIT; -- 提交事务  
    END IF;  
END;  
END;
```

新建一个 sql 窗口使用 CALL 语句激活存储过程，并使用 SELECT 语句查看

```
CALL 借书(7,20081237,2001231);  
SELECT * FROM 借阅;
```

借书流水号为 7，借书证号为 20081237，图书编号为 2001231。

	借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注
1	1	20081237	3007071	2010/09/19	2010/09/20		...
2	2	20071235	1005063	2010/10/20	2011/02/20	1	...
3	3	20071235	2001232	2011/09/01			...
4	4	20061234	1005063	2011/09/20			...
5	5	20051001	3007071	2011/09/10			...
6	6	20071235	1005050	2011/10/20	2012/02/20	1	...
7	7	20081237	2001231	2024/11/12 19:11:35			...

发现添加成功，借书日期为系统时间。

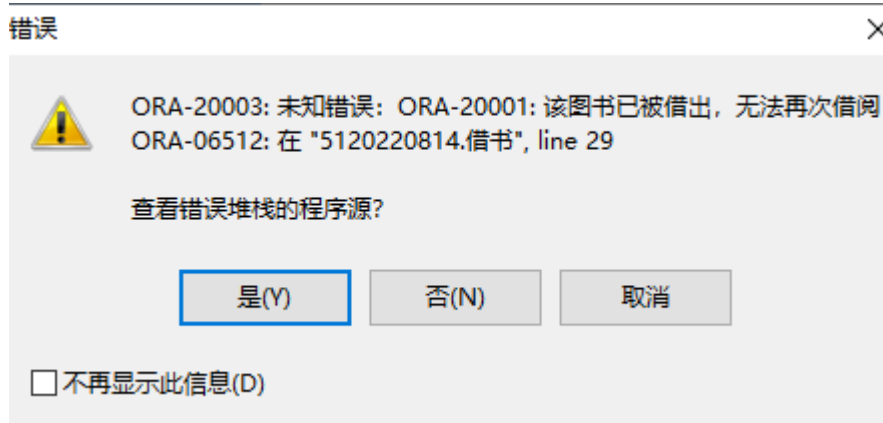
使用 SELECT * FROM 图书；查看图书是否借出是否改为“是”

	图书编号	ISBN	是否借出	备注
1	2001231	7040195836	是	你好222 ...

再次测试相同图书的借书操作 `CALL 借书(8,20071235,2001231);`

借书流水号为8，借书证号为20071235，图书编号为2001231（与上一次操作图书编号一样）。

发现弹出以下错误——“该图书已被借出，无法再次借阅”。



这是因为图书编号为 2001231 的图书已经被借出。在定义存储过程时，我添加了一个 `IF` 条件，用于在执行过程中首先检查图书表中该图书的借出状态。如果该字段的值为“是”，则表示该图书已被借出，不能再次进行借阅操作。此时，存储过程会抛出一个自定义错误提示，提醒用户该图书已被借出，无法再次借阅。

```
IF b_是否借出 = '是' THEN
    RAISE_APPLICATION_ERROR(-20001, '该图书已被借出，无法再次借阅');
```

2、预约功能

建立存储过程完成图书管理系统中的预约功能。

- 预约时要求输入预约流水号，借书证号，ISBN。
- 存储过程先检查输入的 ISBN 版本的图书是否都已借出，如果是则进行预约，否则提示“该书目有可借图书，请查找”。
- 预约时间为系统时间。

```

CREATE OR REPLACE PROCEDURE b_预约(
    b_预约流水号 IN 预约.预约流水号%TYPE,
    b_借书证号 IN 预约.借书证号%TYPE,
    b_ISBN IN 预约.ISBN%TYPE
)
AS
    b_数量 NUMBER;          统计ISBN相同的图书数量，如果为0则可预约
BEGIN
    SELECT COUNT(*) INTO b_数量 FROM 图书
    WHERE 图书.ISBN = b_ISBN AND 图书.是否借出 = '否';
    IF b_数量 = 0 THEN
        INSERT INTO 预约 (预约流水号, 借书证号, ISBN, 预约时间)
        VALUES (b_预约流水号, b_借书证号, b_ISBN, SYSDATE);
        COMMIT;
    ELSE
        dbms_output.put_line('该书目有可借图书，请查找！');
    END IF;
END;

```

使用 CALL b_预约(2, 20081237, 9787508040110);

SELECT * FROM 预约;

实现借书证号“20081237”预约 ISBN 为“9787508040110”的图书
查看预约表，发现添加成功。

	预约流水号	借书证号	ISBN	预约时间
1	1	20081237	9787508040110	2011/09/11
2	2	20081237	9787508040110	2024/11/12 20:10:18

预约存在可借阅图书的图书：

```
CALL b_预约(4, 20081237, 9787506336239);
```

无法添加记录。

3、还书功能

建立存储过程完成图书管理系统中的还书功能。

- 还书时要求输入借书证号，图书编号，罚款分类号
- 还书日期为系统时间。
- 图书的是否借出改为‘否’。

```
CREATE OR REPLACE PROCEDURE 还书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE,
    b_罚款分类号 IN 借阅.罚款分类号%TYPE
)
AS
BEGIN
    UPDATE 借阅
    SET 借阅.归还日期 = SYSDATE,
        借阅.罚款分类号 = b_罚款分类号
    WHERE 借阅.借书证号 = b_借书证号
        AND 借阅.图书编号 = b_图书编号 |
        AND 借阅.归还日期 IS NULL;
    UPDATE 图书 SET 图书.是否借出 = '否'
    WHERE 图书.图书编号 = b_图书编号;
    COMMIT;
END;
```

3个参数

系统时间

更新图书是否借出

使用 CALL 还书(20081237, 2001231, null);
SELECT * FROM 借阅;
实现借书证号“20081237”归还图书编号为“2001231”的图书操作。

	借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注
▶ 1	1	20081237	3007071	2010/09/19	2010/09/20		...
2	2	20071235	1005063	2010/10/20	2011/02/20	1	...
3	3	20071235	2001232	2011/09/01			...
4	4	20061234	1005063	2011/09/20			...
5	5	20051001	3007071	2011/09/10			...
6	6	20071235	1005050	2011/10/20	2012/02/20	1	...
7	7	20081237	2001231	2024/11/12 19:11:35	2024/11/12 20:26:03		...

归还成功。

4、通过序列和触发器实现借阅表中借阅流水号字段自动递增

```
DROP SEQUENCE 借阅流水号序列; 删除之前创建的序列

CREATE SEQUENCE 借阅流水号序列
START WITH 8 -- 起始值
INCREMENT BY 1 -- 每次递增的值
NOCACHE; -- 不缓存序列值（可选）

CREATE OR REPLACE TRIGGER 借阅流水号触发器
BEFORE INSERT ON 借阅
FOR EACH ROW
BEGIN
    -- 使用序列生成唯一的借阅流水号
    :NEW.借阅流水号 := 借阅流水号序列.NEXTVAL;
END;
```

起始值选择 8——之前借阅表已有 7 个借阅记录。

新建一个窗口插入借阅记录：实现借书证号“20071235”借书图书编号为“2001231”的图书操作。

```
INSERT INTO 借阅 (借书证号, 图书编号, 借书日期) VALUES
('20071235', '2001231', SYSDATE);
SELECT * FROM 借阅;
```

	借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注
1	1	20081237	3007071	2010/09/19	2010/09/20		...
2	2	20071235	1005063	2010/10/20	2011/02/20	1	...
3	3	20071235	2001232	2011/09/01			...
4	4	20061234	1005063	2011/09/20			...
5	5	20051001	3007071	2011/09/10			...
6	6	20071235	1005050	2011/10/20	2012/02/20	1	...
7	7	20081237	2001231	2024/11/12 19:11:35	2024/11/12 20:26:03		...
8	8	20071235	2001231	2024/11/12 20:39:08			...

发现虽然未输入借阅流水号，但借阅流水号为 8，实现了借阅流水号的自动递增。

5、修改借书功能的存储过程。

该存储过程要求：

- (1) 借书时输入借书证号，图书编号。（即该函数有 2 个输入参数）
- (2) 借书时，借书日期为系统时间。

*该存储过程主体部分只有 insert into 语句。

```
CREATE OR REPLACE PROCEDURE 借书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE
)
AS
    b_是否借出 图书.是否借出%TYPE;
BEGIN
    SELECT 是否借出 INTO b_是否借出
    FROM 图书
    WHERE 图书.图书编号 = b_图书编号;

    IF b_是否借出 = '是' THEN
        RAISE_APPLICATION_ERROR(-20001, '该图书已被借出，无法再次借阅');
    ELSE
        INSERT INTO 借阅 (借书证号, 图书编号, 借书日期, 归还日期, 罚款分类号, 备注)
        VALUES (b_借书证号, b_图书编号, SYSDATE, NULL, NULL, NULL);
        UPDATE 图书
        SET 图书.是否借出 = '是'
        WHERE 图书.图书编号 = b_图书编号;

        COMMIT;
    END IF;
END;
```

```
执行 INSERT INTO 借阅 (借书证号, 图书编号, 借书日期) VALUES
('20071235', '2001231', SYSDATE);
SELECT * FROM 借阅;
```

```
INSERT INTO 借阅 (借书证号, 图书编号, 借书日期) VALUES ('20071235', '2001231', SYSDATE);
```

Insert 借阅 Select 借阅 Select 图书							
借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注	
1	20081237	3007071	2010/09/19	2010/09/20		...	
2	20071235	1005063	2010/10/20	2011/02/20	1	...	
3	20071235	2001232	2011/09/01			...	
4	20061234	1005063	2011/09/20			...	
5	20051001	3007071	2011/09/10			...	
6	20071235	1005050	2011/10/20	2012/02/20	1	...	
7	20081237	2001231	2024/11/12 19:11:35	2024/11/12 20:26:03		...	
8	20071235	2001231	2024/11/12 20:39:08			...	
9	20071235	2001231	2024/11/12 20:50:52			...	

但是发现是否借出判断，以及是否借出状态更改无反应。

这是因为直接执行 INSERT INTO 语句会绕过借书过程的逻辑，因此不会更新，IF 条件也不会生效。

要执行以下代码才能调用借书过程。（或者 CALL 借书('20071235', '2001231');）

```
BEGIN
    借书(b_借书证号 => '20081237', b_图书编号 => '2001231');
END;
```

借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注	
1	20081237	3007071	2010/09/19	2010/09/20		...	
2	20071235	1005063	2010/10/20	2011/02/20	1	...	
3	20071235	2001232	2011/09/01			...	
4	20061234	1005063	2011/09/20			...	
5	20051001	3007071	2011/09/10			...	
6	20071235	1005050	2011/10/20	2012/02/20	1	...	
7	13	20081237	2001231	2024/11/12 22:39:04		...	

图书状态也更改了。

图书编号	ISBN	是否借出	备注
1	2001231	7040195836	是 你好222 ...
2	1005050	9787506336239	否 ...
3	1005063	9787508040110	是 ...
4	3007071	9787010073750	是 ...
5	2001232	7040195836	是 ...

实验拓展题：

（3）还书时，如果超期（该图书借阅时间超过两个月），那么借阅表中该记录的罚款分类号值为 1。


```

CREATE OR REPLACE PROCEDURE 还书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE
)
AS
    v_借书日期 借阅.借书日期%TYPE;
    v_超期天数 NUMBER; -- 超期天数, 临时变量
BEGIN
    SELECT 借书日期 INTO v_借书日期
    FROM 借阅
    WHERE 借书证号 = b_借书证号
        AND 图书编号 = b_图书编号
        AND 归还日期 IS NULL;

    v_超期天数 := SYSDATE - v_借书日期;
    IF v_超期天数 > 60 THEN
        UPDATE 借阅
        SET 归还日期 = SYSDATE,
            罚款分类号 = 1
        WHERE 借书证号 = b_借书证号
            AND 图书编号 = b_图书编号
            AND 归还日期 IS NULL;
    ELSE
        UPDATE 借阅
        SET 归还日期 = SYSDATE
        WHERE 借书证号 = b_借书证号
            AND 图书编号 = b_图书编号
            AND 归还日期 IS NULL;
    END IF;
    UPDATE 图书
    SET 是否借出 = '否'
    WHERE 图书编号 = b_图书编号;

    COMMIT;
END;

```

使用 CALL 还书(20071235, 2001232); 测试:

	借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注
1	1	20081237	3007071	2010/09/19	2010/09/20		...
2	2	20071235	1005063	2010/10/20	2011/02/20	1	...
3	3	20071235	2001232	2011/09/01	2024/11/12 22:54:45	1	...
4	4	20061234	1005063	2011/09/20			...
5	5	20051001	3007071	2011/09/10			...
6	6	20071235	1005050	2011/10/20	2012/02/20	1	...
7	13	20081237	2001231	2024/11/12 22:39:04			...

图书状态也更新

	图书编号	ISBN	是否借出	备注
1	2001231	7040195836	是	你好222 ...
2	1005050	9787506336239	否	...
3	1005063	9787508040110	是	...
4	3007071	9787010073750	是	...
5	2001232	7040195836	否	...

6、建立与借书存储过程相对应的触发器。

当借阅表中加入借阅信息时，该触发器触发，自动修改所借图书的是否借出改为‘是’。

```
CREATE OR REPLACE TRIGGER 借阅插入触发器
BEFORE INSERT ON 借阅
FOR EACH ROW
BEGIN
    UPDATE 图书
    SET 是否借出 = '是'
    WHERE 图书编号 = :NEW.图书编号;
END;
```

使用 INSERT INTO 借阅 (借书证号, 图书编号, 借书日期) VALUES ('20071235', '2001232', SYSDATE); 插入借阅信息。

	借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注
▶	1	20081237	3007071	2010/09/19	2010/09/20		...
	2	20071235	1005063	2010/10/20	2011/02/20	1	...
	3	20071235	2001232	2011/09/01	2024/11/12 22:54:45	1	...
	4	20061234	1005063	2011/09/20			...
	5	20051001	3007071	2011/09/10			...
	6	20071235	1005050	2011/10/20	2012/02/20	1	...
	7	14	20071235	2001232	2024/11/12 23:01:19	2024/11/12 23:02:36	...
	8	15	20071235	2001232	2024/11/12 23:03:05		...

查看图书状态：

	图书编号	ISBN	是否借出	备注
▶ 1	2001231	7040195836	是	你好222 ...
2	1005050	9787506336239	否	...
3	1005063	9787508040110	是	...
4	3007071	9787010073750	是	...
5	2001232	7040195836	是	...

7、实验拓展题

建立丢失存储过程，如果图书丢失，那么借阅表中该记录的罚款分类号的值为 3，图书表中该书的是否借出为“是”（本来为‘是’，不用修改），备注为“丢失”。

```

CREATE OR REPLACE PROCEDURE 丢失图书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE
)
AS
BEGIN
    UPDATE 借阅
    SET 罚款分类号 = 3, 更新罚款分类号和备注
      备注 = '丢失'
    WHERE 借书证号 = b_借书证号
      AND 图书编号 = b_图书编号
      AND 归还日期 IS NULL; 确保是未归还借阅记录
    COMMIT;
END;

```

使用 CALL 丢失图书(20071235, 2001232);测试:

	借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注
▶ 1	1	20081237	3007071	2010/09/19	2010/09/20		...
2	2	20071235	1005063	2010/10/20	2011/02/20	1	...
3	3	20071235	2001232	2011/09/01	2024/11/12 22:54:45	1	...
4	4	20061234	1005063	2011/09/20			...
5	5	20051001	3007071	2011/09/10			...
6	6	20071235	1005050	2011/10/20	2012/02/20	1	...
7	14	20071235	2001232	2024/11/12 23:01:19	2024/11/12 23:02:36		...
8	15	20071235	2001232	2024/11/12 23:03:05		3	丢失 ...

图书状态:

	图书编号	ISBN	是否借出	备注
▶ 1	2001231	7040195836	是	你好222 ...
2	1005050	9787506336239	否	...
3	1005063	9787508040110	是	...
4	3007071	9787010073750	是	...
5	2001232	7040195836	是	...

4. 实验代码

```

CREATE OR REPLACE PROCEDURE 借书 (
    -- 输入参数
    b_借阅流水号 IN 借阅.借阅流水号%TYPE,
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE
)
AS
    b_是否借出 图书.是否借出%TYPE; -- 临时变量
BEGIN
    SELECT 是否借出 INTO b_是否借出
    FROM 图书
    WHERE 图书编号 = b_图书编号;

```

```

        IF b_是否借出 = '是' THEN
            RAISE_APPLICATION_ERROR(-20001, '该图书已被借出，无法再次借阅');
        ELSE
            INSERT INTO 借阅 (借阅流水号, 借书证号, 图书编号, 借书日期, 归还日期, 罚款分类号, 备注)
            VALUES (b_借阅流水号, b_借书证号, b_图书编号, SYSDATE, NULL, NULL, NULL);
            UPDATE 图书
            SET 是否借出 = '是'
            WHERE 图书编号 = b_图书编号;
            COMMIT;--提交事务
        END IF;
    END;

```

```

CREATE OR REPLACE PROCEDURE b_预约(
    b_预约流水号 IN 预约.预约流水号%TYPE,
    b_借书证号 IN 预约.借书证号%TYPE,
    b_ISBN IN 预约.ISBN%TYPE
)
AS
    b_数量 NUMBER;
BEGIN
    SELECT COUNT(*) INTO b_数量 FROM 图书
    WHERE 图书.ISBN = b_ISBN AND 图书.是否借出 = '否';
    IF b_数量 = 0 THEN
        INSERT INTO 预约 (预约流水号, 借书证号, ISBN, 预约时间)
        VALUES (b_预约流水号, b_借书证号, b_ISBN, SYSDATE);
        COMMIT;
    ELSE
        dbms_output.put_line('该书目有可借图书，请查找! ');
    END IF;
END;

```

```

CREATE OR REPLACE PROCEDURE 还书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE,
    b_罚款分类号 IN 借阅.罚款分类号%TYPE
)
AS
BEGIN
    UPDATE 借阅
    SET 借阅.归还日期 = SYSDATE,

```

```

        借阅.罚款分类号 = b_罚款分类号
WHERE 借阅.借书证号 = b_借书证号
      AND 借阅.图书编号 = b_图书编号
      AND 借阅.归还日期 IS NULL;
UPDATE 图书 SET 图书.是否借出 = '否'
WHERE 图书.图书编号 = b_图书编号;
COMMIT;
END;

```

```

DROP SEQUENCE 借阅流水号序列;
CREATE SEQUENCE 借阅流水号序列
START WITH 8          -- 起始值
INCREMENT BY 1        -- 每次递增的值
NOCACHE;              -- 不缓存序列值
CREATE OR REPLACE TRIGGER 借阅流水号触发器
BEFORE INSERT ON 借阅
FOR EACH ROW
BEGIN
    -- 使用序列生成唯一的借阅流水号
    :NEW.借阅流水号 := 借阅流水号序列.NEXTVAL;
END;

```

```

CREATE OR REPLACE PROCEDURE 借书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE
)
AS
    b_是否借出 图书.是否借出%TYPE;
BEGIN
    SELECT 是否借出 INTO b_是否借出
    FROM 图书
    WHERE 图书.图书编号 = b_图书编号;

    IF b_是否借出 = '是' THEN
        RAISE_APPLICATION_ERROR(-20001, '该图书已被借出，无法再次借阅');
    ELSE
        INSERT INTO 借阅 (借书证号, 图书编号, 借书日期, 归还日期, 罚款分
        类号, 备注)
        VALUES (b_借书证号, b_图书编号, SYSDATE, NULL, NULL, NULL);
        UPDATE 图书
        SET 图书.是否借出 = '是'
        WHERE 图书.图书编号 = b_图书编号;
    END IF;
END;

```

```
        COMMIT;
    END IF;
END;
```

```
CREATE OR REPLACE PROCEDURE 还书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE
)
AS
```

```
    v_借书日期 借阅.借书日期%TYPE;
    v_超期天数 NUMBER;
```

```
BEGIN
```

```
    SELECT 借书日期 INTO v_借书日期
    FROM 借阅
    WHERE 借书证号 = b_借书证号
        AND 图书编号 = b_图书编号
        AND 归还日期 IS NULL;
```

```
    v_超期天数 := SYSDATE - v_借书日期;
    IF v_超期天数 > 60 THEN
```

```
        UPDATE 借阅
        SET 归还日期 = SYSDATE,
            罚款分类号 = 1
        WHERE 借书证号 = b_借书证号
            AND 图书编号 = b_图书编号
            AND 归还日期 IS NULL;
```

```
    ELSE
```

```
        UPDATE 借阅
        SET 归还日期 = SYSDATE
        WHERE 借书证号 = b_借书证号
            AND 图书编号 = b_图书编号
            AND 归还日期 IS NULL;
```

```
    END IF;
```

```
    UPDATE 图书
    SET 是否借出 = '否'
    WHERE 图书编号 = b_图书编号;
```

```
    COMMIT;
END;
```

```
CREATE OR REPLACE TRIGGER 借阅插入触发器
```

```

BEFORE INSERT ON 借阅
FOR EACH ROW
BEGIN
    UPDATE 图书
    SET 是否借出 = '是'
    WHERE 图书编号 = :NEW.图书编号;
END;

```

```

CREATE OR REPLACE PROCEDURE 丢失图书 (
    b_借书证号 IN 借阅.借书证号%TYPE,
    b_图书编号 IN 借阅.图书编号%TYPE
)
AS
BEGIN
    UPDATE 借阅
    SET 罚款分类号 = 3,
        备注 = '丢失'
    WHERE 借书证号 = b_借书证号
        AND 图书编号 = b_图书编号
        AND 归还日期 IS NULL;
    COMMIT;
END;

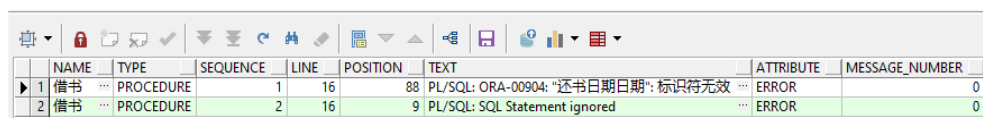
```

5. 实验结果（结果在第 3 部分）

问题：

1. 查看编译错误

创建存储过程过程中，可能会出现语法问题，导致编译出错，但是只有在 CALL 语句激活时才知道出问题，并且不知道问题在哪里，可以新创建一个窗口输入 SELECT * FROM USER_ERRORS WHERE NAME = '借书'；，就可以查看到查看存储过程的编译错误信息：



	NAME	TYPE	SEQUENCE	LINE	POSITION	TEXT	ATTRIBUTE	MESSAGE_NUMBER
1	借书	PROCEDURE	1	16	88	PL/SQL: ORA-00904: "还书日期日期": 标识符无效	ERROR	0
2	借书	PROCEDURE	2	16	9	PL/SQL: SQL Statement ignored	ERROR	0

如图是还书日期处参数名打错，应该为归还日期。

2. DBMS_OUTPUT.PUT_LINE 的使用

如果在存储过程中使用了 DBMS_OUTPUT.PUT_LINE 来调试，在应用程序中无法看到输出，可以通过存储过程返回值或者通过 OUT 参数返回消息。

也可以使用 `RAISE_APPLICATION_ERROR` 触发一个自定义错误。`RAISE_APPLICATION_ERROR` 会引发一个自定义错误,并且可以给出一个负的错误码和一个描述性错误消息。通过这种方式,应用程序可以捕捉到异常,并相应地处理错误信息。

例如: `RAISE_APPLICATION_ERROR(-20001, '该图书已被借出,无法再次借阅');`

3. 两个存储过程定义不能在同一个 SQL 窗口中

存储过程的名称不能与数据库中已存在的对象(如表、视图、其他存储过程等)名称重复。如果发现命名冲突,可以重新命名存储过程。

4. 在 INSERT INTO 中不调用存储过程

在 Oracle 数据库中,存储过程和 `INSERT INTO` 语句有不同的调用机制。`INSERT INTO` 语句本身是直接执行的数据操作语句,而存储过程是封装了一系列操作的代码块。

`CALL` 语句是用来显式调用存储过程的语句。在调用存储过程时,Oracle 会执行存储过程中的所有逻辑,包括 `INSERT` 语句、更新操作或其他任何操作。

而 `INSERT INTO` 无法直接在语句中嵌入存储过程的调用,因为它们的执行机制不同。如果在执行 `INSERT` 操作时希望能够执行存储过程中的逻辑,需要建立触发器。触发器能对 `INSERT`、`UPDATE` 或 `DELETE` 操作进行自动响应

6. 实验心得

在本次实验中,我深入了解了存储过程、触发器及 `INSERT INTO` 操作的关系和区别。通过创建和调用存储过程,我学会了如何封装复杂的业务逻辑,并通过 `CALL` 语句触发执行。在存储过程中使用 `RAISE_APPLICATION_ERROR` 可以帮助在发生错误时及时反馈信息,提高了调试的效率。

实验还涉及了触发器的使用,它能在数据表中插入数据时自动执行特定操作,比如更新图书的借出状态。这种自动化处理使得数据库操作更加高效和一致。

通过这些操作,我体会到数据库管理中存储过程和触发器的重要性,尤其是在复杂的业务场景中,它们能帮助简化代码,减少重复操作,并保证数据的完整性和一致性。此外,掌握如何处理存储过程中的错误和调试信息也为后续的开发提供了很多便利。